

Kata: Konvertera romerska nummer

Vad är: En kata är en programmeringsövning som skrivs under en sittning för att öva upp färdigheter i kodskrivande och logiskt tänkande. Ordet är lånat från japanskan där det syftar till kampsportskoreografi. Det norpades på oklara grunder av Dave Thomas när han skrev boken *The Pragmatic Programmer* (1999). Eftersom det lät mystiskt och coolt fick det snabbt fäste bland programmerare (som ofta är mkt ocoola och är i stort behov av mer coolhet).

Problembeskrivning

Katan är en känd programmeringsövning som går ut på att skriva en metod för att konvertera decimalnummer till romerska nummer. Romerska nummer använder kombinationer av bokstäver från det latinska alfabetet för att beteckna värden. De baseras på följande sju symboler:

I	1
V	5
X	10
L	50
C	100
D	500
M	1000

Nummer formas genom att kombinera symboler och addera deras värden. Oftast placeras symboler i enlighet med de värden de representerar: de börjar med största värdet först. (Precis som vi i decimal sätter den största siffran längst till vänster: 1111 i bas-10 betyder $1000 + 100 + 10 + 1$).

När mindre värden står skrivna framför större värden i en romersk sekvens innebär det att det mindre värdet ska subtraheras från det större: IX betyder exempelvis 9 eftersom $X - I = 10 - 1$.

Nummer	Beräkning	Värde	Kommentar
III	$1 + 1 + 1$	3	Enbart addition utförs
DCX	$500 + 100 + 10$	610	Enbart addition
DCIX	$500 + 100 + (10 - 1)$	609	Addition och subtraktion

Uppgift

Skapa en algoritm som konverterar decimaltal till romerska nummer genom Test-Driven Development. Algoritmen behöver bara kunna hantera decimaltal upp till tusen (M). Ni får jobba ensamma eller i grupper om två eller flera.

Instruktioner

Börja med att skapa en klass:

```
public class RomanNumeralConverter
{
    public static string convert(int number)
    {
        //din kod här
    }
}
```

Skapa sedan en testklass till den här klassen. När du satt upp din testklass är du redo för att påbörja övningen.

Att göra:

1. Börja med att skriva ett test för siffran 1.
2. Kör testet (det ska misslyckas och bli rött).
3. Skriv minsta möjliga kod för att få det att bli grönt (godkänt).
4. Upprepa igen för siffran 2.
5. Refaktorer koden vid behov så att den blir mer generaliserad: tanken är inte att vi ska ha tusen test för tusen nummer, utan att vi ska kunna hantera många av dem på samma sätt.

Om du fastnar:

* Ägna 5-10 minuter åt att fundera över hur du kan lösa ett steg, men sitt inte fast för länge på samma steg.

* Om du inte lyckas lösa ett problem så fråga läraren eller en annan student om hjälp, eller fråga en chatbot - men säg åt den att inte hjälpa dig med hela uppgiften i så fall utan enbart det aktuella testet, så förstör den inte utmaningen (idén är att du ska öva på att arbeta inkrementellt även om du inte finner rätt lösning direkt).

Exempel på hur det första testet skulle kunna se ut:

```
@Test
void shouldConvertToI()
{
    String result = RomanNumeralConverter.convert(1);
    assertEquals("I", result);
}
```

Hårdkodning är okej för att komma igång! Det lättaste sättet att få det här testet att bli godkänt är att hårdkoda lösningen (men ganska snart måste vi börja refaktorerar):

```
if (number == 1) { return "I"; }
else { return ""; }
```

Att tänka på

Hur många tester behövs?

Det här beror självklart på hur koden man skapar kommer att se ut, men mellan 10-20 tester bör vara tillräckligt för att täcka in alla nummer upp till 1000 (M). Skulle du behöva fler är det dock inga problem!

I den här övningen behöver du inte skriva negativa test för varje case (men du får om ni vill!).

Idén med Test-Driven Development är inte att testa varje enskilt nummer, och inte heller att algoritmen vi skapar ska bli en ren lookup-table där vi gör en if-check för varje decimalnummer och sedan returnerar motsvarande romersk siffra.

Tanken är att vi ska börja väldigt simpelt och sedan förfina koden för att bli mer generell, men vi måste göra det i små inkrementella steg: När vi skrivit test för de första tre talen, kan vi exempelvis göra logiken mer generell så att vi printar dem med en loop? Osv.

Titta inte för långt framåt i förväg. Fokusera på det aktuella testet: förhastad optimering är en av de dominerande anledningarna till att kod går sönder och projekt misslyckas. Hittar du ingen bra lösning för stunden? Hårdkoda vad metoden returnerar för det aktuella fallet. Vi kan alltid refaktorera senare.

Vad test egentligen gör i TDD

Det är inte antalet test i sig som är viktigt, utan att:

- * De flesta test vi gör introducerar en ny regel (ok att tumma på detta i början!)
- * Att varje test gör designen lite bättre
- * Att inga gamla tester går sönder när vi introducerar nya

Om ett gammalt test går sönder måste vi åtgärda koden så att det passerar: vi får INTE ta bort eller förändra själva testet.

Refaktorering

Att refaktorera kod innebär att vi gör den mer robust, men också tydligare för oss själva och andra. Tänk på att använda en tydlig variabelnamnsättning: det blir snabbt svårt att navigera genom kod som inte är tydlig med vad den gör, särskilt i takt med att man introducerar fler regler och undantag till algoritmen.

Det är också helt ok att skapa **privata** hjälpmetoder inuti RomanNumeralConverter för att bryta ut logik om ni känner att det gör koden tydligare och lättare att resonera kring. De ska dock vara just privata i så fall, så att ingen t.ex. kan skriva test för dem från utsidan: den enda metod som bör ha åtkomst till dem är convert().

Lycka till! 